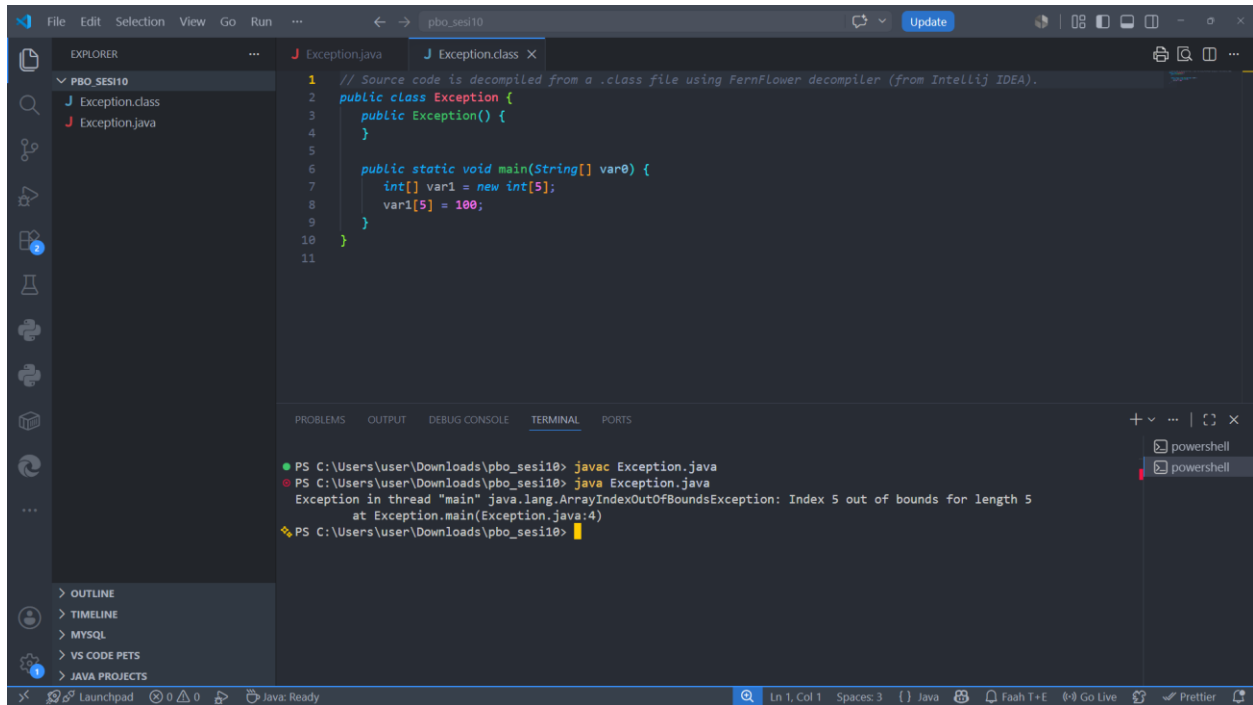


Nama : Ajeng Mirani=ti
PBO TI23A
Percobaan 1

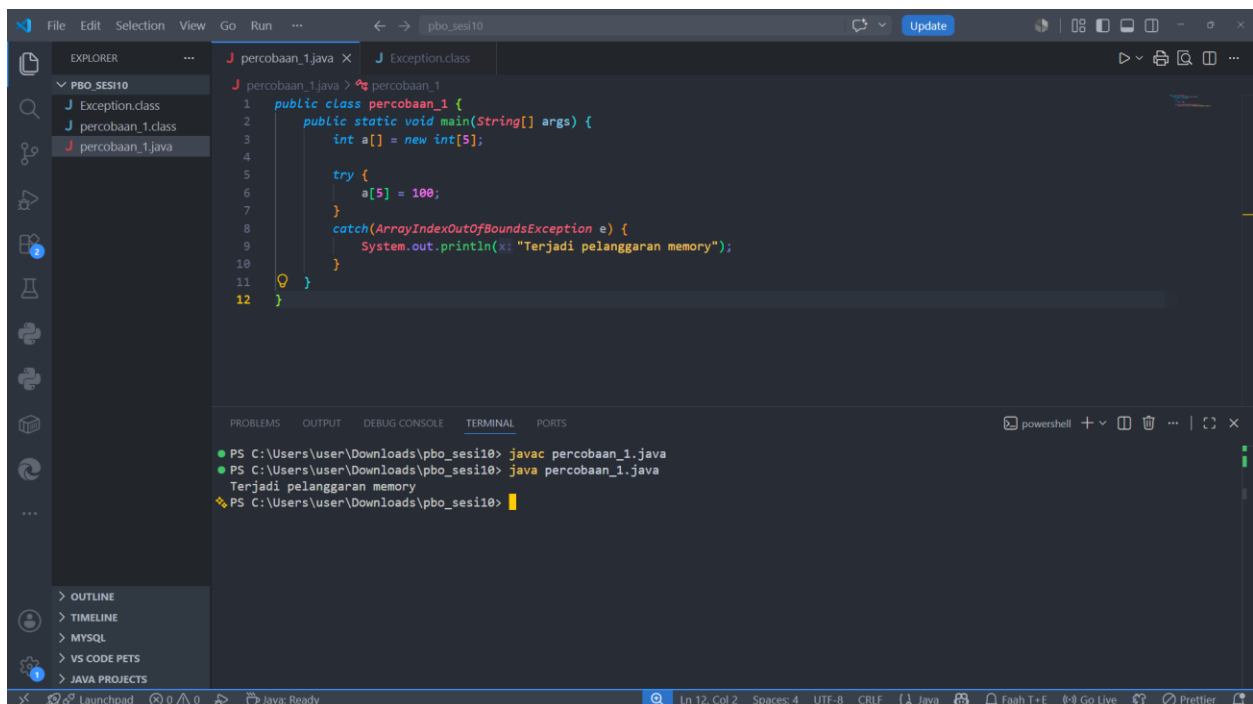
Error



The screenshot shows the Visual Studio Code editor with a project named 'pbo_sesi10'. The Explorer panel on the left shows the project structure with 'Exception.class' and 'Exception.java'. The main editor displays the code for 'Exception.java', which is a decompiled class. The code defines a 'public class Exception' with an empty constructor and a 'main' method. The 'main' method declares an integer array 'var1' of size 5 and attempts to assign the value 100 to 'var1[5]'. The bottom panel shows the 'TERMINAL' output, which contains the following commands and error message:

```
PS C:\Users\user\Downloads\pbo_sesi10> javac Exception.java
PS C:\Users\user\Downloads\pbo_sesi10> java Exception.java
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5
    at Exception.main(Exception.java:4)
PS C:\Users\user\Downloads\pbo_sesi10>
```

Pembetulan

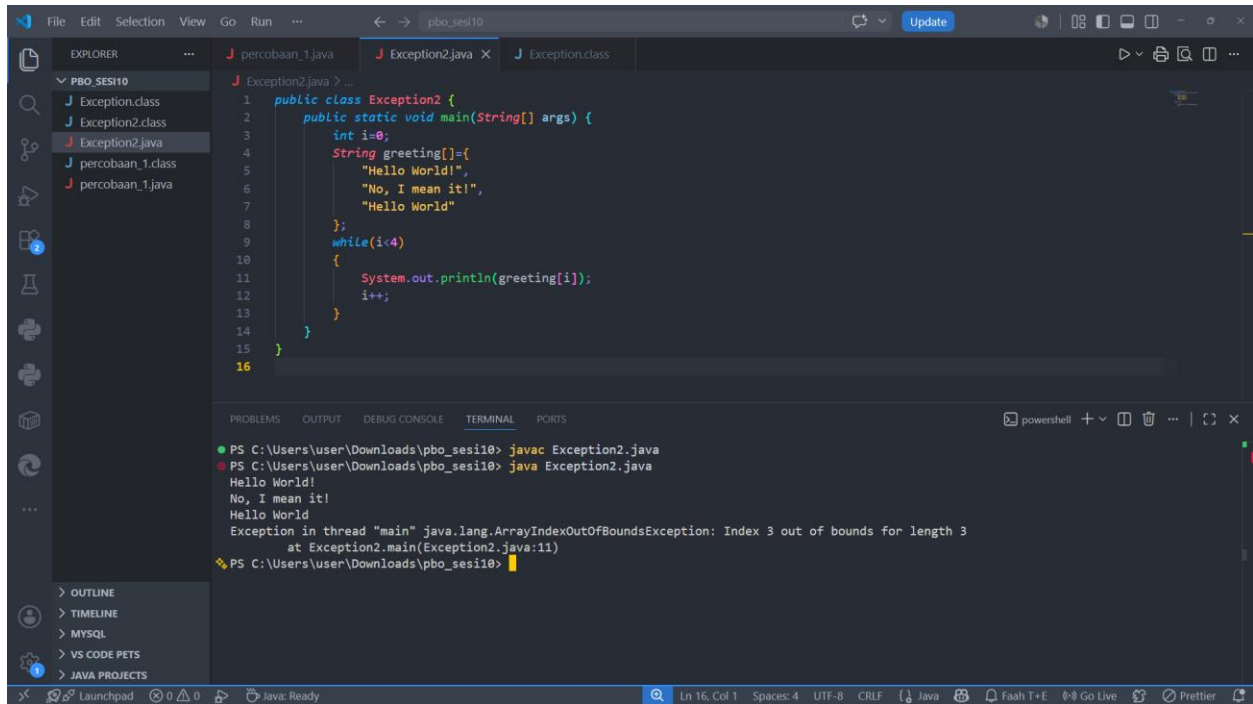


The screenshot shows the Visual Studio Code editor with the same project 'pbo_sesi10'. The Explorer panel now shows 'percobaan_1.class' and 'percobaan_1.java'. The main editor displays the code for 'percobaan_1.java', which is a corrected version of the previous code. The code defines a 'public class percobaan_1' with a 'main' method. The 'main' method declares an integer array 'a' of size 5 and attempts to assign the value 100 to 'a[5]'. The code is wrapped in a 'try-catch' block to handle the 'ArrayIndexOutOfBoundsException'. The bottom panel shows the 'TERMINAL' output, which contains the following commands and output:

```
PS C:\Users\user\Downloads\pbo_sesi10> javac percobaan_1.java
PS C:\Users\user\Downloads\pbo_sesi10> java percobaan_1.java
Terjadi pelanggaran memory
PS C:\Users\user\Downloads\pbo_sesi10>
```

Percobaan 2

Error



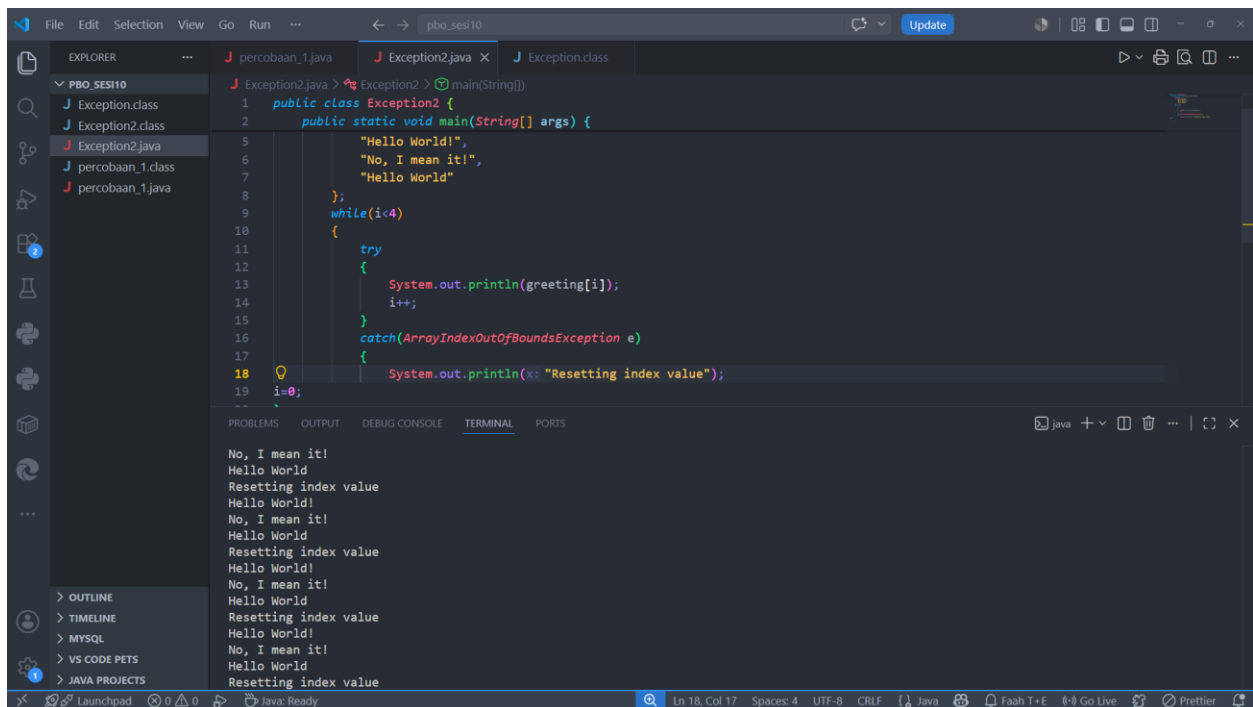
The screenshot shows the VS Code editor with a Java file named `Exception2.java`. The code is as follows:

```
1 public class Exception2 {
2     public static void main(String[] args) {
3         int i=0;
4         String greeting[]={
5             "Hello World!",
6             "No, I mean it!",
7             "Hello World"
8         };
9         while(i<4)
10        {
11            System.out.println(greeting[i]);
12            i++;
13        }
14    }
15 }
16
```

The terminal output shows the execution of the program, which results in an `ArrayIndexOutOfBoundsException` at line 11:

```
PS C:\Users\user\Downloads\pbo_sesi10> javac Exception2.java
PS C:\Users\user\Downloads\pbo_sesi10> java Exception2.java
Hello World!
No, I mean it!
Hello World
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
    at Exception2.main(Exception2.java:11)
PS C:\Users\user\Downloads\pbo_sesi10>
```

Pembetulan



The screenshot shows the VS Code editor with the same Java file, but now with a try-catch block to handle the exception. The code is as follows:

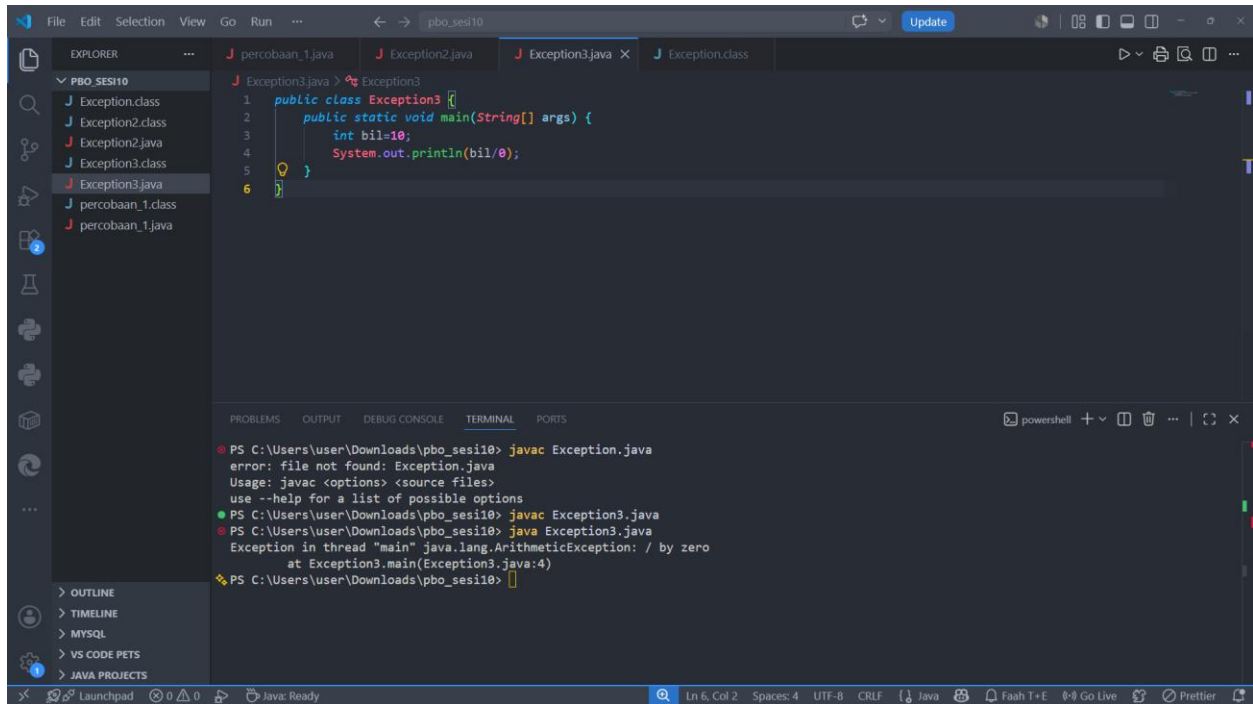
```
1 public class Exception2 {
2     public static void main(String[] args) {
3         int i=0;
4         String greeting[]={
5             "Hello World!",
6             "No, I mean it!",
7             "Hello World"
8         };
9         while(i<4)
10        {
11            try
12            {
13                System.out.println(greeting[i]);
14                i++;
15            }
16            catch(ArrayIndexOutOfBoundsException e)
17            {
18                System.out.println("Resetting index value");
19                i=0;
20            }
21        }
22    }
23 }
```

The terminal output shows the execution of the program, which now handles the exception and prints "Resetting index value" before continuing the loop:

```
No, I mean it!
Hello World
Resetting index value
Hello World!
No, I mean it!
Hello World
Resetting index value
Hello World!
No, I mean it!
Hello World
Resetting index value
Hello World!
No, I mean it!
Hello World
Resetting index value
```

Percobaan 3

Error



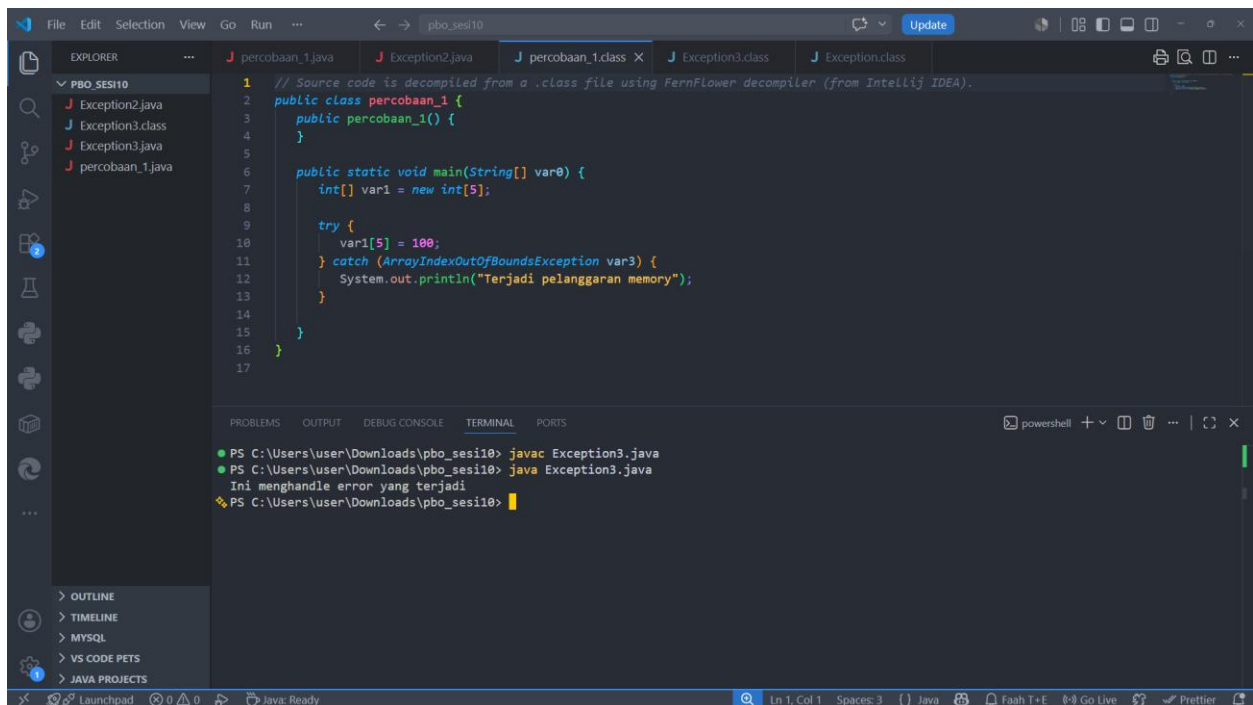
The screenshot shows the VS Code interface with a Java project named 'pbo_sesi10'. The Explorer panel on the left shows the project structure. The main editor displays the code for 'Exception3.java'. The code is as follows:

```
1 public class Exception3 {  
2     public static void main(String[] args) {  
3         int bil=10;  
4         System.out.println(bil/0);  
5     }  
6 }
```

The bottom panel shows the 'TERMINAL' output, which contains the following commands and error messages:

```
PS C:\Users\user\Downloads\pbo_sesi10> javac Exception.java  
error: file not found: Exception.java  
Usage: javac <options> <source files>  
use --help for a list of possible options  
PS C:\Users\user\Downloads\pbo_sesi10> javac Exception3.java  
PS C:\Users\user\Downloads\pbo_sesi10> java Exception3.java  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
at Exception3.main(Exception3.java:4)  
PS C:\Users\user\Downloads\pbo_sesi10>
```

Perbaikan



The screenshot shows the VS Code interface with the same project. The Explorer panel now shows 'percobaan_1.class' as the active file. The main editor displays the code for 'percobaan_1.java'. The code is as follows:

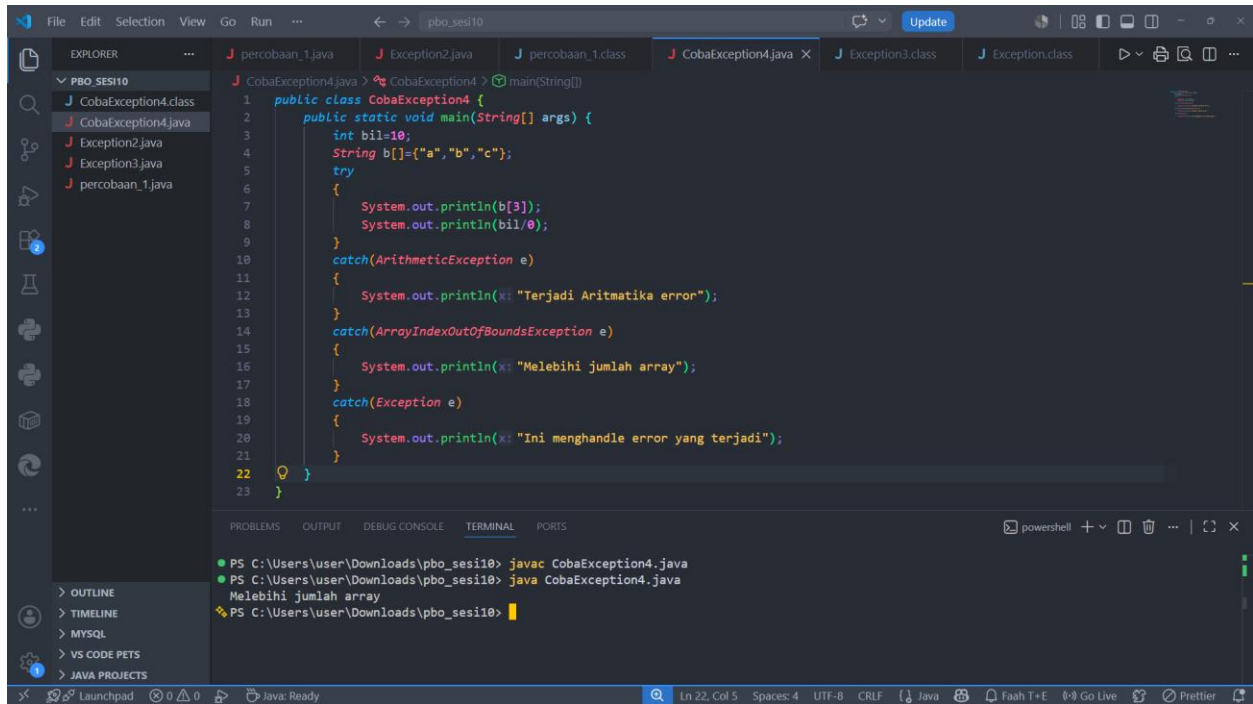
```
1 // Source code is decompiled from a .class file using FernFlow decompiler (from IntelliJ IDEA).  
2 public class percobaan_1 {  
3     public percobaan_1() {  
4     }  
5  
6     public static void main(String[] var0) {  
7         int[] var1 = new int[5];  
8  
9         try {  
10            var1[5] = 100;  
11        } catch (ArrayIndexOutOfBoundsException var3) {  
12            System.out.println("Terjadi pelanggaran memory");  
13        }  
14    }  
15 }  
16  
17 }
```

The bottom panel shows the 'TERMINAL' output, which contains the following commands and messages:

```
PS C:\Users\user\Downloads\pbo_sesi10> javac Exception3.java  
PS C:\Users\user\Downloads\pbo_sesi10> java Exception3.java  
Ini menghandle error yang terjadi  
PS C:\Users\user\Downloads\pbo_sesi10>
```

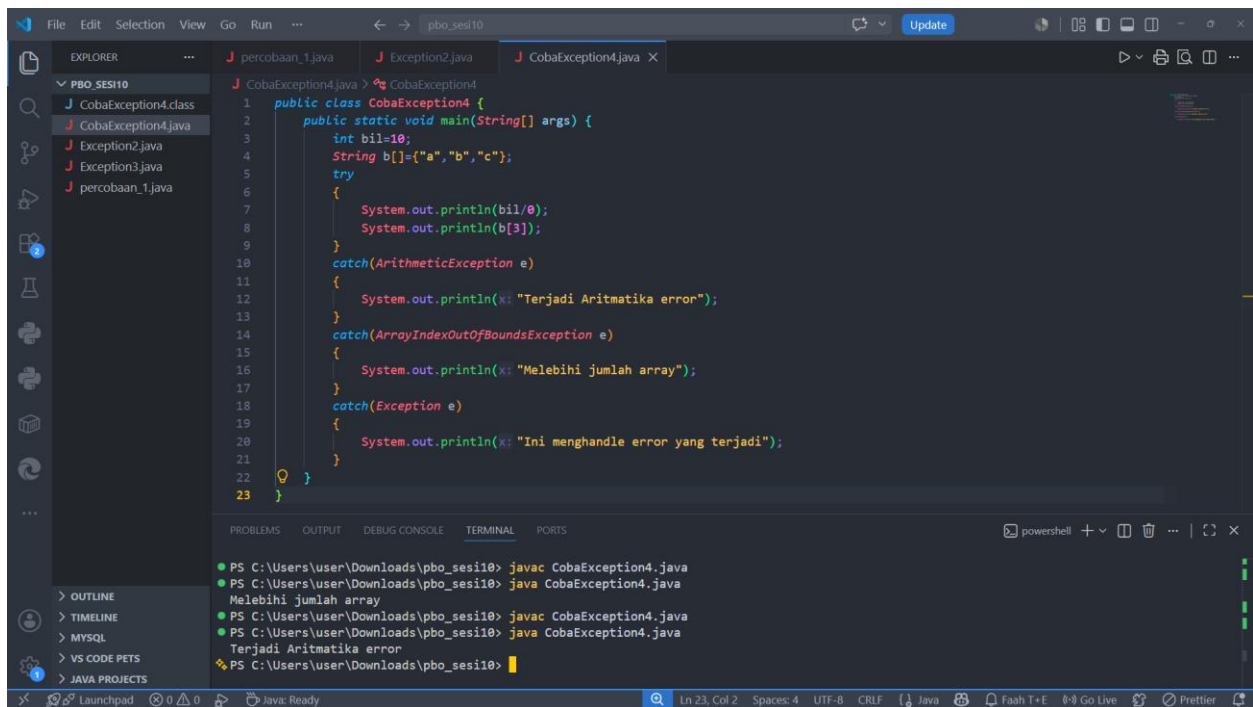
Percobaan 4

Error



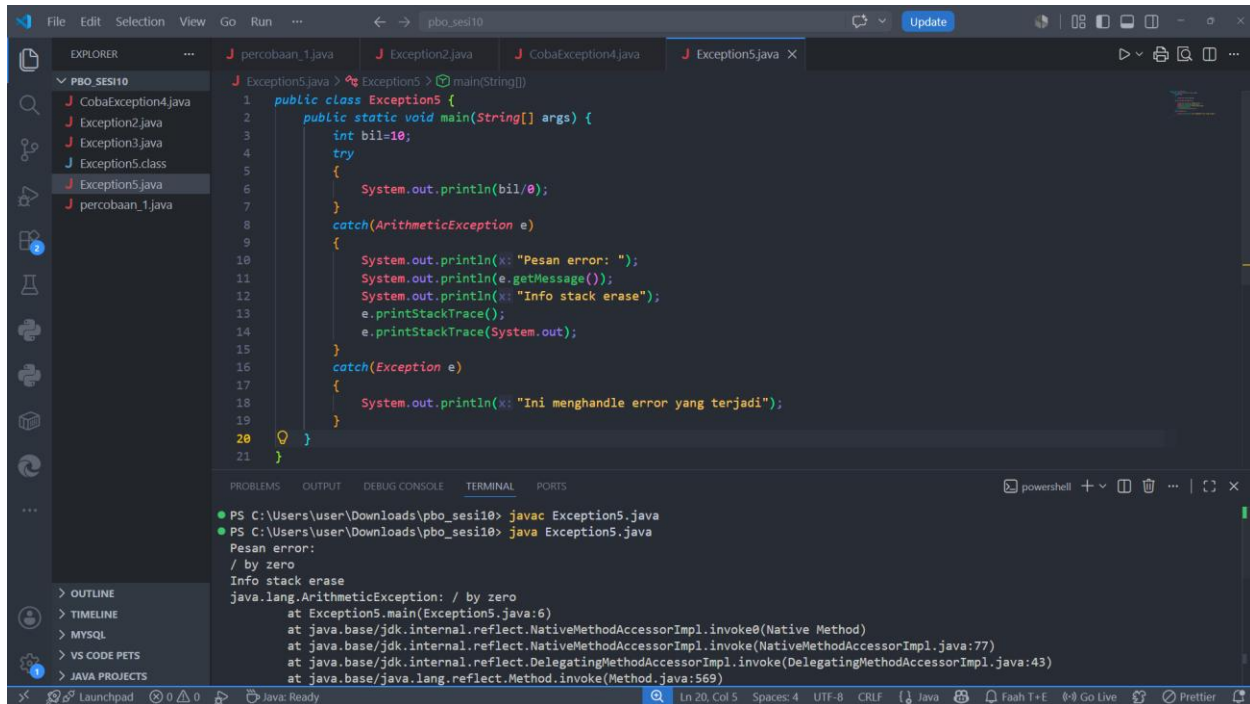
```
File Edit Selection View Go Run ... pbo_sesi10
EXPLORER
PBO_SESI10
CobaException4.class
CobaException4.java
Exception2.java
Exception3.java
percobaan_1.java
CobaException4.java
1 public class CobaException4 {
2     public static void main(String[] args) {
3         int bil=10;
4         String b[]{"a","b","c"};
5         try
6         {
7             System.out.println(b[3]);
8             System.out.println(bil/0);
9         }
10        catch(ArithmeticException e)
11        {
12            System.out.println("Terjadi Aritmatika error");
13        }
14        catch(ArrayIndexOutOfBoundsException e)
15        {
16            System.out.println("Melebihi jumlah array");
17        }
18        catch(Exception e)
19        {
20            System.out.println("Ini menghandle error yang terjadi");
21        }
22    }
23 }
TERMINAL
PS C:\Users\user\Downloads\pbo_sesi10> javac CobaException4.java
PS C:\Users\user\Downloads\pbo_sesi10> java CobaException4.java
Melebihi jumlah array
PS C:\Users\user\Downloads\pbo_sesi10>
```

Pembetulan



```
File Edit Selection View Go Run ... pbo_sesi10
EXPLORER
PBO_SESI10
CobaException4.class
CobaException4.java
Exception2.java
Exception3.java
percobaan_1.java
CobaException4.java
1 public class CobaException4 {
2     public static void main(String[] args) {
3         int bil=10;
4         String b[]{"a","b","c"};
5         try
6         {
7             System.out.println(bil/0);
8             System.out.println(b[3]);
9         }
10        catch(ArithmeticException e)
11        {
12            System.out.println("Terjadi Aritmatika error");
13        }
14        catch(ArrayIndexOutOfBoundsException e)
15        {
16            System.out.println("Melebihi jumlah array");
17        }
18        catch(Exception e)
19        {
20            System.out.println("Ini menghandle error yang terjadi");
21        }
22    }
23 }
TERMINAL
PS C:\Users\user\Downloads\pbo_sesi10> javac CobaException4.java
PS C:\Users\user\Downloads\pbo_sesi10> java CobaException4.java
Melebihi jumlah array
PS C:\Users\user\Downloads\pbo_sesi10> javac CobaException4.java
PS C:\Users\user\Downloads\pbo_sesi10> java CobaException4.java
Terjadi Aritmatika error
PS C:\Users\user\Downloads\pbo_sesi10>
```

Percobaan 5



The screenshot shows a Java IDE with a project named 'PBO_SESI10'. The Explorer panel on the left lists files: 'CobaException4.java', 'Exception2.java', 'Exception3.java', 'Exception5.class', 'Exception5.java', and 'percobaan_1.java'. The main editor displays the code for 'Exception5.java':

```
1 public class Exception5 {
2     public static void main(String[] args) {
3         int bil=10;
4         try
5         {
6             System.out.println(bil/0);
7         }
8         catch(ArithmeticException e)
9         {
10            System.out.println("Pesan error: ");
11            System.out.println(e.getMessage());
12            System.out.println("Info stack erase");
13            e.printStackTrace();
14            e.printStackTrace(System.out);
15        }
16        catch(Exception e)
17        {
18            System.out.println("Ini menghandle error yang terjadi");
19        }
20    }
21 }
```

The bottom panel shows the output of running the program. It displays the error message 'Pesan error: / by zero' and the stack trace for 'java.lang.ArithmeticException: / by zero'.

```
PS C:\Users\user\Downloads\pbo_sesi10> javac Exception5.java
PS C:\Users\user\Downloads\pbo_sesi10> java Exception5.java
Pesan error:
/ by zero
Info stack erase
java.lang.ArithmeticException: / by zero
    at Exception5.main(Exception5.java:6)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
    at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
    at java.base/java.lang.reflect.Method.invoke(Method.java:569)
```

Pada percobaan 5, blok try digunakan untuk menjalankan kode yang berpotensi menghasilkan error, yaitu operasi pembagian bilangan dengan angka nol ($bil/0$). Ketika terjadi error `ArithmeticException`, program tidak berhenti secara paksa karena error tersebut ditangkap oleh blok `catch`. Method `getMessage()` digunakan untuk menampilkan pesan error, sedangkan `printStackTrace()` digunakan untuk menampilkan informasi lokasi terjadinya error pada program. Penggunaan try-catch membuat program dapat menangani kesalahan saat runtime.

Percobaan 6

The screenshot displays an IDE interface with the following components:

- Explorer Panel (Left):** Lists files in the project: `pbo_sesi10`, `Cobaxception4.java`, `Exception2.java`, `Exception3.java`, `Exception5.class`, `Exception5.java`, `percobaan_1.java`, `ThrowExample.class`, and `ThrowExample.java`.
- Editor (Center):** Shows the code for `ThrowExample.java`:


```

1  public class ThrowExample {
2      static void demo()
3      {
4          NullPointerException t;
5          t=new NullPointerException("Coba Throw");
6          throw t;
7
8          // Baris ini tidak lagi dikerjakan;
9          //System.out.println("Ini tidak lagi dicetak");
10
11     }
12     public static void main(String[] args) {
13         try
14         {
15             demo();
16             System.out.println("Selesai");
17         }
18         catch(NullPointerException e)
19         {
20             System.out.println("Ada pesan error: "+e);
21         }
22     }

```
- Problems Panel (Bottom):** Displays error messages:
 - PS C:\Users\user\Downloads\pbo_sesi10> javac ThrowExample.java
 - PS C:\Users\user\Downloads\pbo_sesi10> java ThrowExample.java
 - error: can't find main(String[]) method in class: ThrowExample
 - PS C:\Users\user\Downloads\pbo_sesi10> javac ThrowExample.java
 - PS C:\Users\user\Downloads\pbo_sesi10> java ThrowExample.java
 - Ada pesan error: java.lang.NullPointerException: Coba Throw
 - PS C:\Users\user\Downloads\pbo_sesi10>

Pada program ini digunakan konsep throw dan catch untuk menangani exception secara manual. Method demo() membuat objek NullPointerException dengan pesan "Coba Throw", kemudian melempar exception tersebut menggunakan perintah throw t.

Ketika method `demo()` dipanggil di dalam blok `try`, exception yang dilempar akan menghentikan proses program pada method tersebut sehingga perintah:

```
System.out.println("Selesai");
```

tidak dijalankan.

Exception tersebut kemudian ditangkap oleh blok `catch(NullPointerException e)`. Bagian `catch` menampilkan pesan error menggunakan variabel `e`, sehingga program tidak berhenti secara tiba-tiba tetapi dapat menangani kesalahan yang terjadi.

Kesimpulannya, penggunaan try-catch berfungsi untuk menangkap exception yang dilempar oleh throw, sehingga error dapat dikontrol dan program tetap berjalan dengan baik.

Percobaan 7

The screenshot shows the VS Code editor with the following content:

EXPLORER

- PROJ_SESI10
 - ThrowExample2.java
 - CobaException4.java
 - Exception2.java
 - Exception3.java
 - Exception5.java
 - percobaan_1.java
 - ThrowExample.java
 - ThrowExample2.class
 - ThrowExample2.java

ThrowExample2.java

```

1  public class ThrowExample2 {
2      public static void main(String[] args) {
3          try
4          {
5              throw new Exception(message: "Here's my Exception");
6          }
7          catch(Exception e)
8          {
9              System.out.println("Caught Exception");
10             System.out.println("e.getMessage(): " + e.getMessage());
11             System.out.println("e.toString(): " + e.toString());
12             System.out.println("e.printStackTrace():");
13             e.printStackTrace();
14         }
15     }
16 }

```

PROBLEMS

- PS C:\Users\User\Downloads\pbo_sesi10> javac ThrowExample2.java
- PS C:\Users\User\Downloads\pbo_sesi10> java ThrowExample2.java

OUTPUT

```

Caught Exception
e.getMessage(): Here's my Exception
e.toString(): java.lang.Exception: Here's my Exception
e.printStackTrace():
java.lang.Exception: Here's my Exception
    at ThrowExample2.main(ThrowExample2.java:5)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
    at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
    at java.base/java.lang.reflect.Method.invoke(Method.java:569)
    atjdk.compiler/com.sun.tools.javac.launcher.Main.execute(Main.java:419)
    atjdk.compiler/com.sun.tools.javac.launcher.Main.run(Main.java:192)
    atjdk.compiler/com.sun.tools.javac.launcher.Main.main(Main.java:132)

```

Pada percobaan ini digunakan perintah throw untuk membuat dan melempar exception secara manual menggunakan objek Exception. Exception tersebut dibuat dengan pesan "Here's my Exception" kemudian ditangkap oleh blok catch(Exception e).

Method getMessage() digunakan untuk mengambil pesan error yang terdapat pada exception, sehingga menampilkan:

Here's my Exception

Method `toString()` menampilkan nama tipe exception beserta pesan error, yaitu:

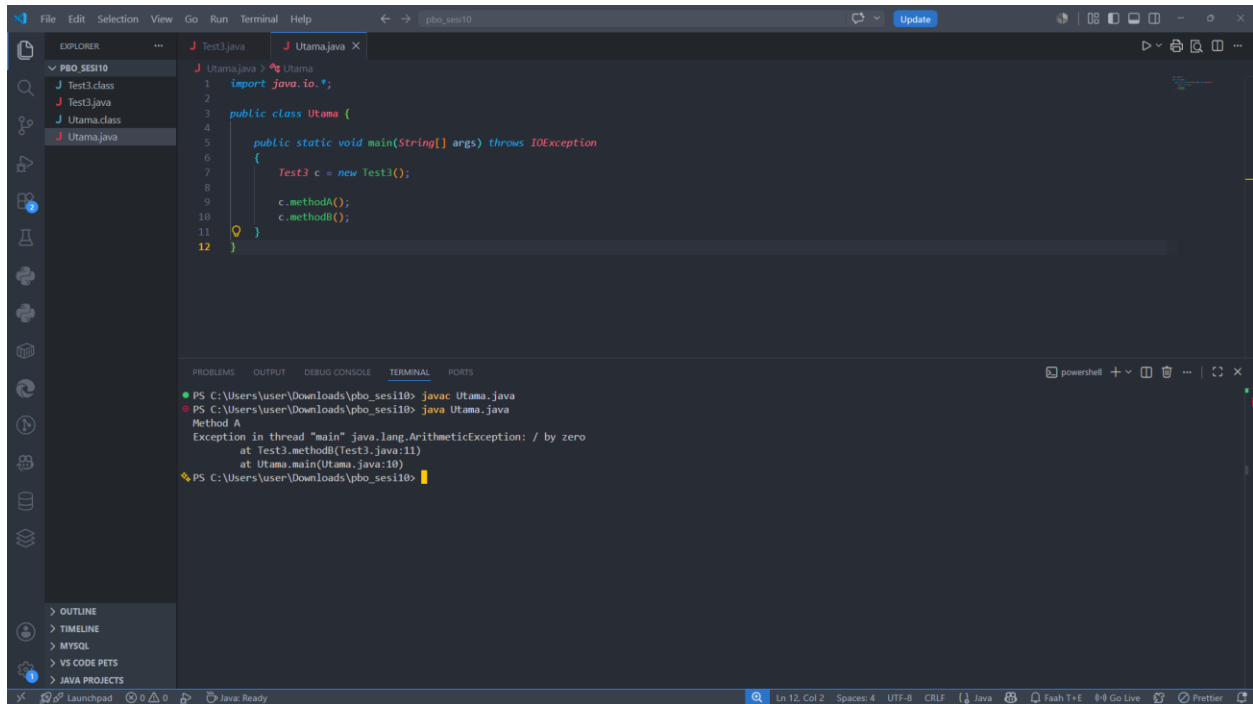
java.lang.Exception: Here's my Exception

Sedangkan `printStackTrace()` digunakan untuk menampilkan informasi detail mengenai lokasi terjadinya exception, termasuk nama class dan nomor baris kode yang menyebabkan error.

Dengan penggunaan try-catch, exception yang dilempar tidak membuat program berhenti secara tiba-tiba, tetapi dapat ditangani dan ditampilkan informasinya kepada pengguna.

Percobaan 8

Method A



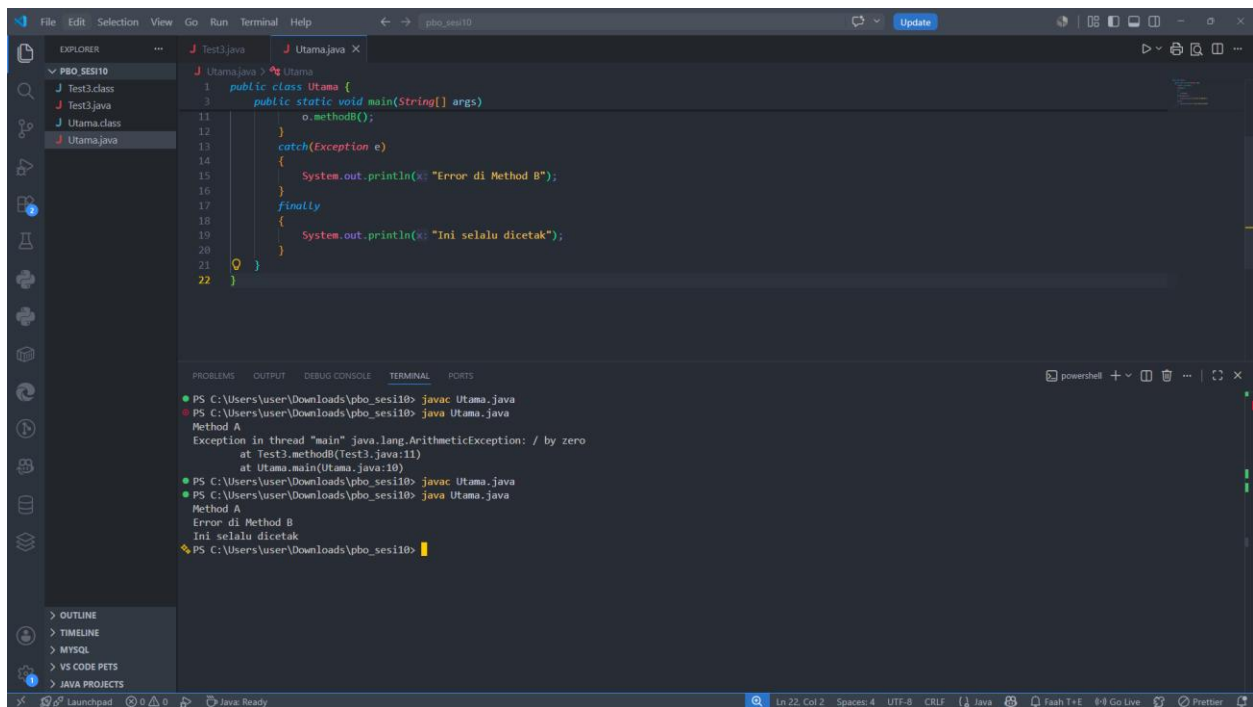
```
File Edit Selection View Go Run Terminal Help
pbo_sesi10

EXPLORER
PBO_SESI10
  Test3.class
  Test3.java
  Utama.class
  Utama.java

Termin
  Utama.java
    1 import java.io.*;
    2
    3
    4 public class Utama {
    5     public static void main(String[] args) throws IOException
    6     {
    7         Test3 c = new Test3();
    8
    9         c.methodA();
   10         c.methodB();
   11     }
   12 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\user\Downloads\pbo_sesi10> javac Utama.java
PS C:\Users\user\Downloads\pbo_sesi10> java Utama.java
Method A
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Test3.methodB(Test3.java:11)
    at Utama.main(Utama.java:10)
PS C:\Users\user\Downloads\pbo_sesi10>
```

Method B



```
File Edit Selection View Go Run Terminal Help
pbo_sesi10

EXPLORER
PBO_SESI10
  Test3.class
  Test3.java
  Utama.class
  Utama.java

Termin
  Utama.java
    1 import java.io.*;
    2
    3 public class Utama {
    4     public static void main(String[] args)
    5     {
    6         o.methodB();
    7     }
    8     catch(Exception e)
    9     {
   10         System.out.println(k: "Error di Method B");
   11     }
   12     finally
   13     {
   14         System.out.println(k: "Ini selalu dicetak");
   15     }
   16 }
   17
   18
   19
   20
   21
   22 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\user\Downloads\pbo_sesi10> javac Utama.java
PS C:\Users\user\Downloads\pbo_sesi10> java Utama.java
Method A
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Test3.methodB(Test3.java:11)
    at Utama.main(Utama.java:10)
PS C:\Users\user\Downloads\pbo_sesi10> javac Utama.java
PS C:\Users\user\Downloads\pbo_sesi10> java Utama.java
Method A
Error di Method B
Ini selalu dicetak
PS C:\Users\user\Downloads\pbo_sesi10>
```


Pada percobaan ini digunakan konsep **throws**, **try-catch**, dan **finally** dalam penanganan exception.

Pada method `methodB()`, terdapat deklarasi:

`public void methodB() throws IOException`

Deklarasi `throws` digunakan untuk memberi informasi bahwa method tersebut dapat menghasilkan exception yang harus ditangani oleh pemanggil method. Pada program ini, meskipun tertulis `throws IOException`, error yang terjadi sebenarnya adalah `ArithmeticException` karena program melakukan operasi pembagian dengan angka nol:

`System.out.println(20/0);`

Pada program utama versi pertama, exception diteruskan menggunakan `throws IOException` pada method `main()`. Ketika terjadi error, program berhenti karena tidak ada blok `catch` yang menangkap exception tersebut.

Pada program utama versi kedua, pemanggilan `methodB()` dimasukkan ke dalam blok:

```
try  
{  
    o.methodB();  
}  
catch(Exception e)  
{  
    System.out.println("Error di Method B");  
}  
finally  
{  
    System.out.println("Ini selalu dicetak");  
}
```

Blok `try` digunakan untuk menjalankan kode yang kemungkinan menghasilkan error. Jika terjadi exception, blok `catch` akan menangkap error tersebut dan menjalankan perintah:

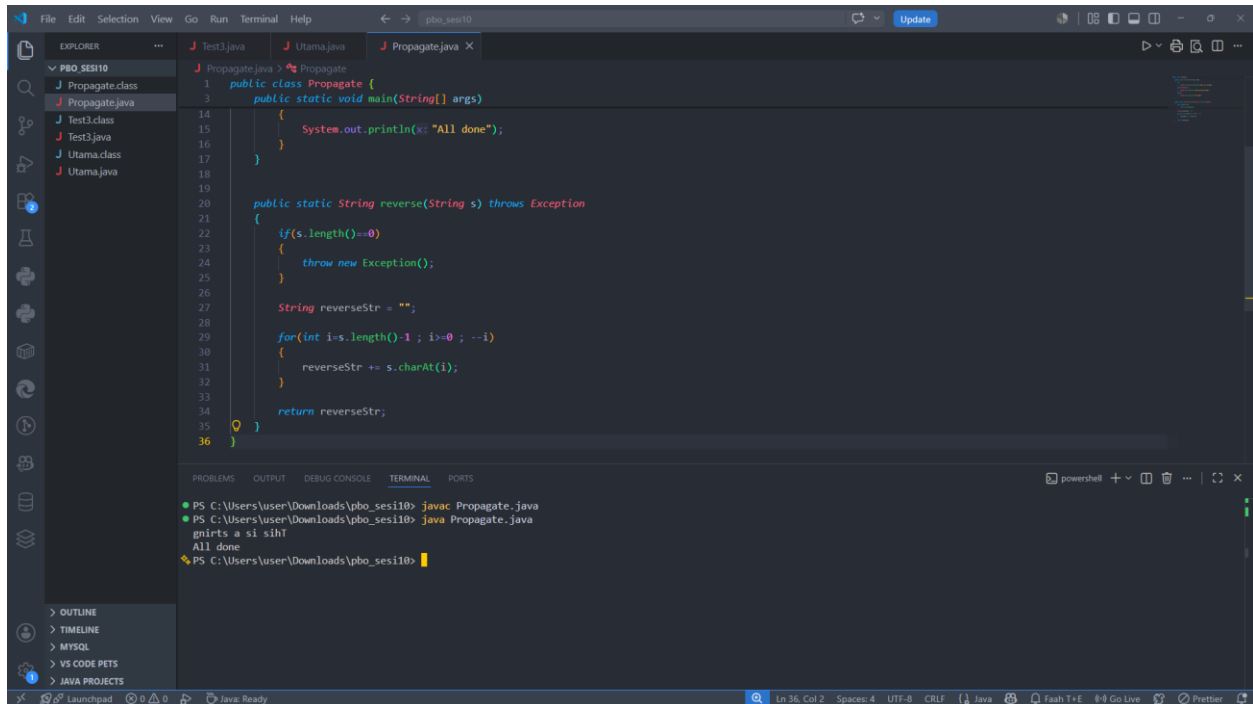
Error di Method B

Kemudian blok finally tetap dijalankan meskipun terjadi error maupun tidak, sehingga menghasilkan output:

Ini selalu dicetak

Kesimpulannya, penggunaan throws digunakan untuk meneruskan kemungkinan exception kepada pemanggil method, sedangkan try-catch digunakan untuk menangani exception agar program tidak berhenti secara langsung. Blok finally digunakan untuk menjalankan perintah yang harus tetap dilakukan setelah proses try-catch selesai.

Percobaan 9



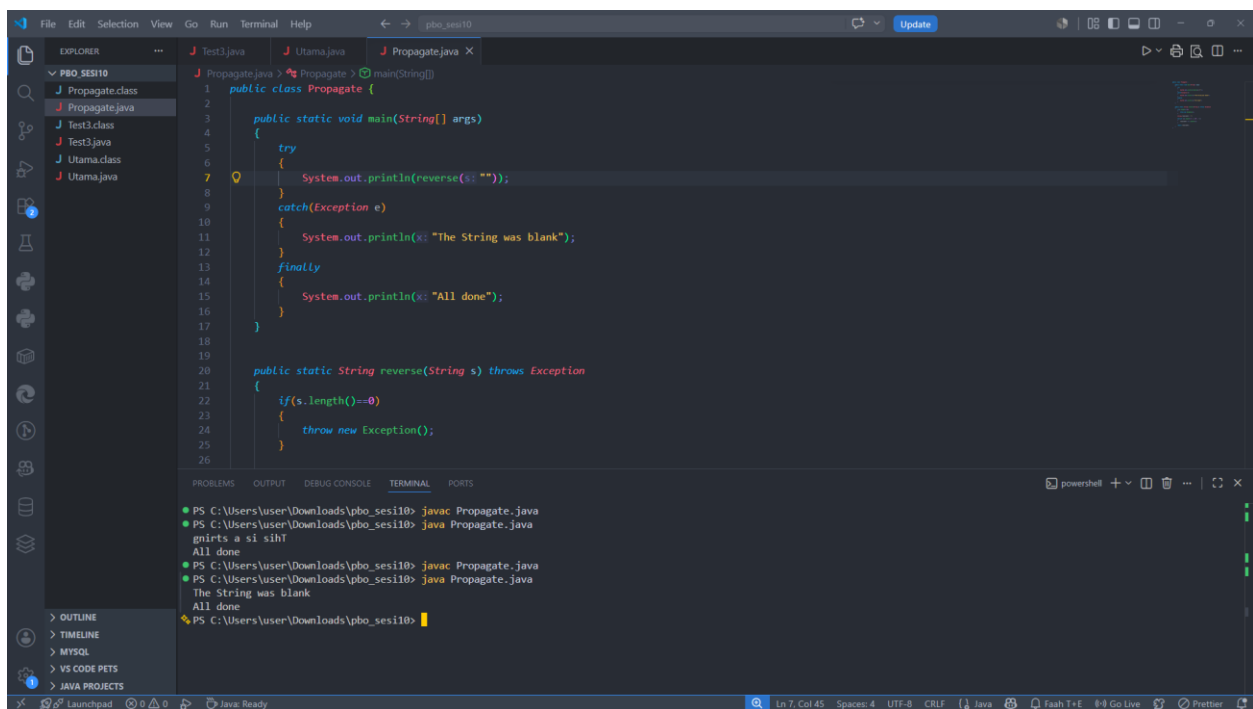
```
File Edit Selection View Go Run Terminal Help
PBO_SESI10
Propagate.java
Test3.java
Utama.java
Propagate.java
Test3.class
Test3.java
Utama.class
Utama.java

1 public class Propagate {
2     public static void main(String[] args)
3     {
4         System.out.println(x: "All done");
5     }
6 }

7 public static String reverse(String s) throws Exception
8 {
9     if(s.length()==0)
10    {
11        throw new Exception();
12    }
13
14    String reverseStr = "";
15
16    for(int i=s.length()-1; i>=0; --i)
17    {
18        reverseStr += s.charAt(i);
19    }
20
21    return reverseStr;
22 }
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\user\Downloads\pbo_sesi10> javac Propagate.java
PS C:\Users\user\Downloads\pbo_sesi10> java Propagate.java
gnirts a si sihT
All done
PS C:\Users\user\Downloads\pbo_sesi10>
```

Sudah diubah:



```
File Edit Selection View Go Run Terminal Help
PBO_SESI10
Propagate.java
Test3.java
Utama.java
Propagate.java
Test3.class
Test3.java
Utama.class
Utama.java

1 public class Propagate {
2     public static void main(String[] args)
3     {
4         try
5         {
6             System.out.println(reverse(s: ""));
7         }
8         catch(Exception e)
9         {
10            System.out.println(x: "The String was blank");
11        }
12        finally
13        {
14            System.out.println(x: "All done");
15        }
16    }
17 }
18
19 public static String reverse(String s) throws Exception
20 {
21     if(s.length()==0)
22     {
23         throw new Exception();
24     }
25 }
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\user\Downloads\pbo_sesi10> javac Propagate.java
PS C:\Users\user\Downloads\pbo_sesi10> java Propagate.java
gnirts a si sihT
All done
PS C:\Users\user\Downloads\pbo_sesi10> javac Propagate.java
PS C:\Users\user\Downloads\pbo_sesi10> java Propagate.java
The String was blank
All done
PS C:\Users\user\Downloads\pbo_sesi10>
```

Pada percobaan ini digunakan konsep **propagate exception** dengan menggunakan throws, throw, try-catch, dan finally.

Method reverse() menggunakan throws Exception untuk meneruskan kemungkinan error kepada method yang memanggilnya. Jika string yang diberikan kosong, maka program menjalankan:

throw new Exception();

sehingga exception dilempar ke blok catch pada method main().

Blok catch berfungsi menangkap exception tersebut dan menampilkan pesan:

The String was blank

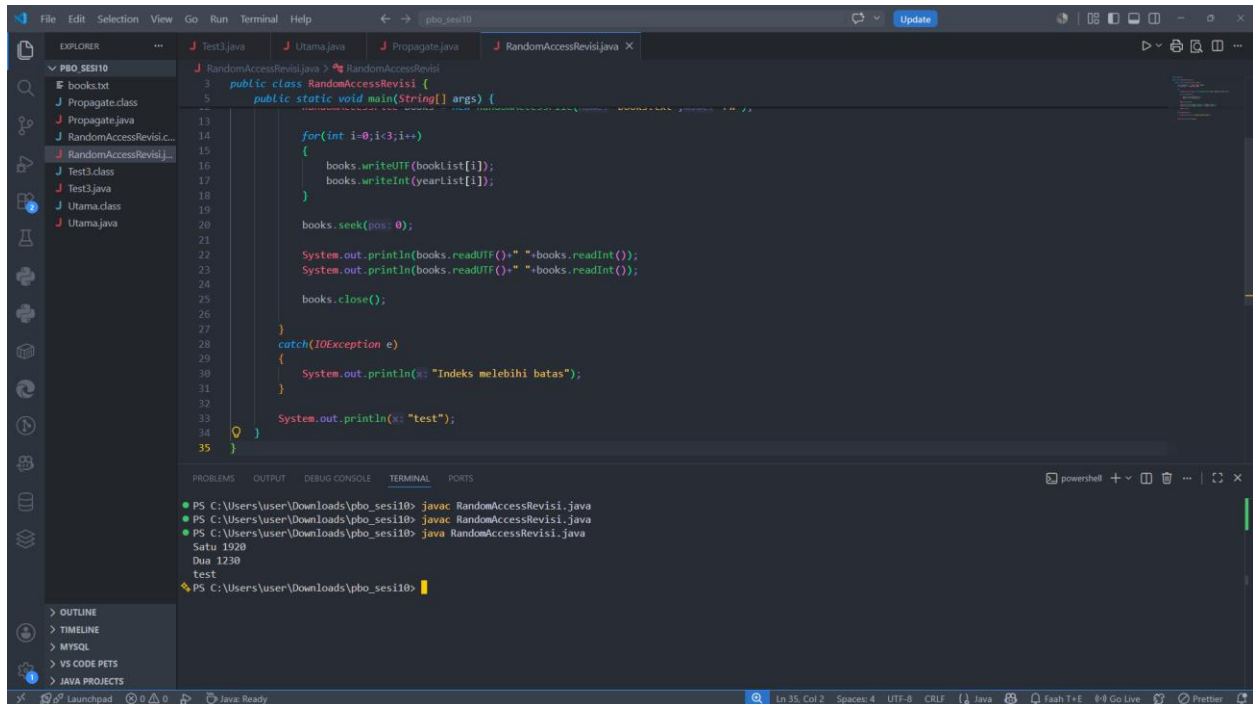
Sedangkan blok finally selalu dijalankan baik terjadi error maupun tidak, sehingga pesan:

All done

tetap muncul.

Kesimpulannya, mekanisme throws dan throw digunakan untuk meneruskan serta membuat exception, sedangkan try-catch-finally digunakan untuk menangani error agar program tetap berjalan dengan baik.

Percobaan 10



```
File Edit Selection View Go Run Terminal Help
pbo_sesi10
Update
EXPLORER
pbo_sesi10
books.txt
Propagate.class
Propagate.java
RandomAccessRevisi.c...
RandomAccessRevisi.j...
Test3.class
Test3.java
Utama.class
Utama.java
RandomAccessRevisi.java
3 public class RandomAccessRevisi {
5     public static void main(String[] args) {
13
14         for(int i=0; i<3; i++)
15         {
16             books.writeUTF(bookList[i]);
17             books.writeInt(yearList[i]);
18         }
19
20         books.seek(books.length());
21
22         System.out.println(books.readUTF()+" "+books.readInt());
23         System.out.println(books.readUTF()+" "+books.readInt());
24
25         books.close();
26
27     }
28     catch(IOException e)
29     {
30         System.out.println("Indeks melebihi batas");
31     }
32
33     System.out.println("test");
34 }
35 }

TERMINAL
PS C:\Users\user\Downloads\pbo_sesi10> javac RandomAccessRevisi.java
PS C:\Users\user\Downloads\pbo_sesi10> javac RandomAccessRevisi.java
PS C:\Users\user\Downloads\pbo_sesi10> java RandomAccessRevisi.java
Dua 1230
test
PS C:\Users\user\Downloads\pbo_sesi10>
```

Pada percobaan ini digunakan class `RandomAccessFile` untuk membuat, menulis, dan membaca file teks secara langsung. Blok `try` digunakan untuk menjalankan proses file yang berpotensi menghasilkan error, seperti membuka file, menulis data, dan membaca data.

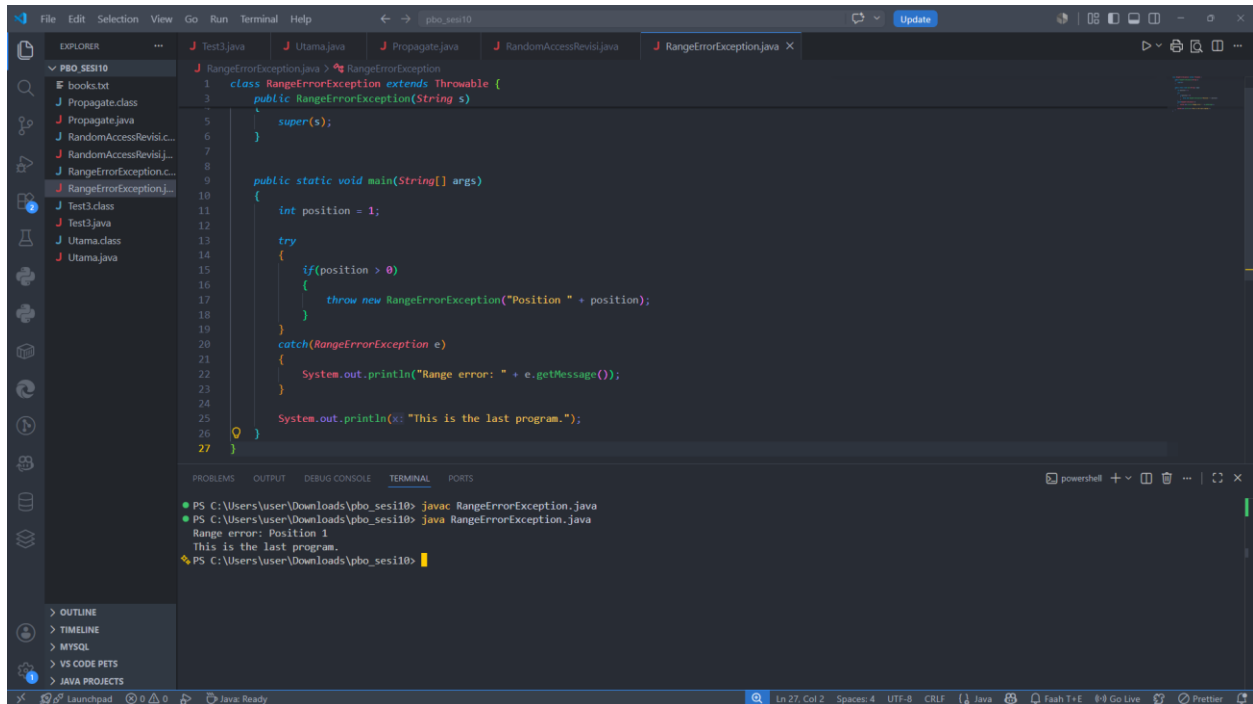
Jika terjadi kesalahan saat proses file, maka exception `IOException` akan ditangkap oleh blok `catch` dan menampilkan pesan:

Indeks melebihi batas

Penggunaan `seek(0)` berfungsi untuk mengembalikan posisi pointer file ke awal sebelum proses pembacaan data dilakukan. Setelah proses selesai, method `close()` digunakan untuk menutup file.

Kesimpulannya, penggunaan `try-catch` pada operasi file membantu menangani kemungkinan kesalahan input/output sehingga program tidak berhenti secara tiba-tiba.

Percobaan 11



```
File Edit Selection View Go Run Terminal Help
RangeErrorException.java
1 class RangeErrorException extends Throwable {
2     public RangeErrorException(String s)
3     {
4         super(s);
5     }
6
7     public static void main(String[] args)
8     {
9         int position = 1;
10
11         try
12         {
13             if(position > 0)
14             {
15                 throw new RangeErrorException("Position " + position);
16             }
17         }
18         catch(RangeErrorException e)
19         {
20             System.out.println("Range error: " + e.getMessage());
21         }
22
23         System.out.println("This is the last program.");
24     }
25 }
26
27

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\User\Downloads\pbo_sesi10> javac RangeErrorException.java
PS C:\Users\User\Downloads\pbo_sesi10> java RangeErrorException.java
Range error: Position 1
This is the last program.
PS C:\Users\User\Downloads\pbo_sesi10>
```

Pada percobaan ini dibuat exception sendiri dengan membuat class `RangeErrorException` yang mewarisi class `Throwable`.

Perintah:

`throw new RangeErrorException("Position " + position);`

digunakan untuk melempar exception secara manual ketika nilai `position` lebih besar dari 0.

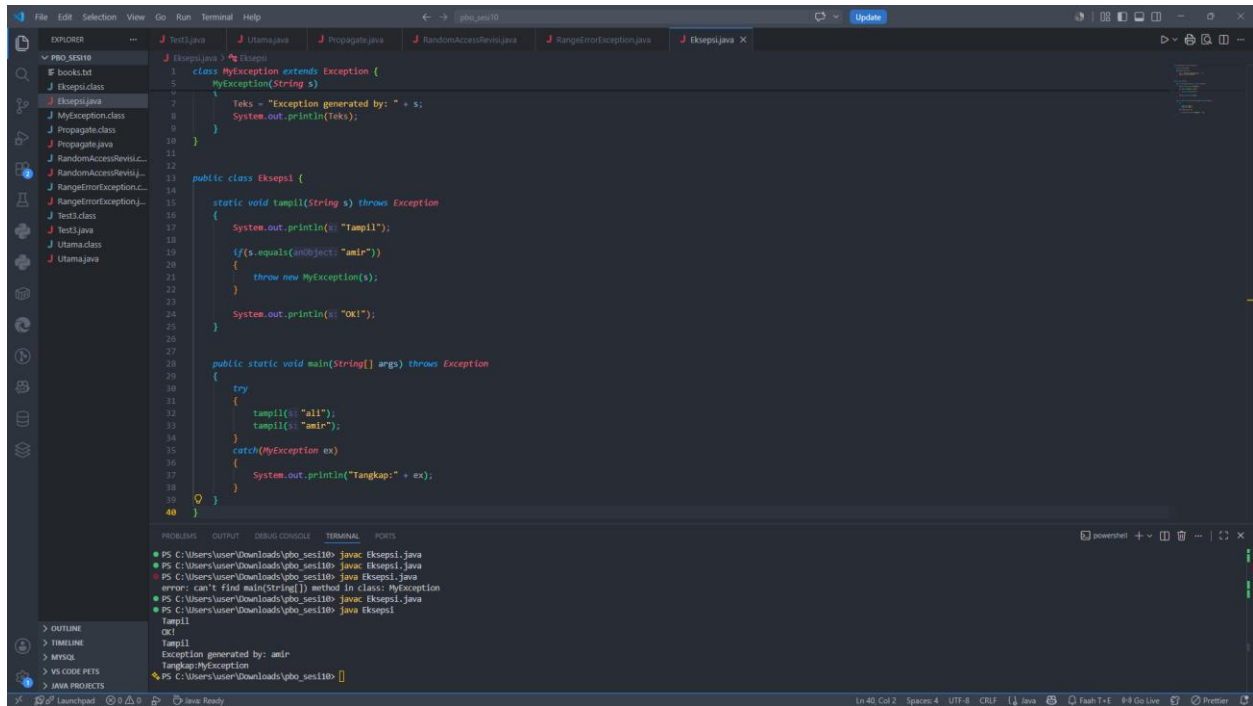
Exception tersebut kemudian ditangkap oleh blok:

`catch(RangeErrorException e)`

yang akan menampilkan pesan error menggunakan method `getMessage()`.

Kesimpulannya, Java memungkinkan pengguna membuat exception sendiri sesuai kebutuhan program dengan cara melakukan `extends` terhadap `Throwable`, kemudian menangkapnya menggunakan `try-catch`.

Percobaan 12



```
1 class MyException extends Exception {
2     MyException(String s) {
3         teks = "Exception generated by: " + s;
4         System.out.println(teks);
5     }
6 }
7
8 public class Eksepsi {
9
10     static void tampil(String s) throws Exception
11     {
12         System.out.println(s);
13         if(s.equals("amir"))
14         {
15             throw new MyException(s);
16         }
17         System.out.println(s);
18     }
19
20     public static void main(String[] args) throws Exception
21     {
22         try
23         {
24             tampil("amir");
25             tampil("amir");
26         }
27         catch(MyException ex)
28         {
29             System.out.println("Tangkap: " + ex);
30         }
31     }
32 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\user\Downloads\gbo_ses18> javac Eksepsi.java
- PS C:\Users\user\Downloads\gbo_ses18> java Eksepsi.java
- PS C:\Users\user\Downloads\gbo_ses18> java Eksepsi.java
- error: can't find main(String[]) method in class: MyException
- PS C:\Users\user\Downloads\gbo_ses18> javac Eksepsi.java
- PS C:\Users\user\Downloads\gbo_ses18> java Eksepsi

Tampil
amir
Exception generated by: amir
Tangkap: MyException

Pada percobaan ini dibuat exception sendiri dengan membuat class `MyException` yang merupakan turunan dari class `Exception`.

Method `tampil()` menggunakan:

throws Exception

untuk meneruskan kemungkinan exception kepada method yang memanggilnya.

Ketika nilai parameter yang diberikan adalah "amir", program menjalankan:

throw new MyException(s);

sehingga objek `MyException` dilempar dan ditangkap oleh blok:

catch(MyException ex)

Program dapat menangani exception tersebut tanpa berhenti secara tiba-tiba.

Kesimpulannya, custom exception menggunakan `extends Exception` dapat digunakan untuk membuat jenis error sendiri sesuai kebutuhan program, kemudian ditangani menggunakan mekanisme `try-catch`.